DAA Assignment 2 (CO2)

Nature of Assignment: Individual

Duration: 1 Week

Total Marks: 50

Submission Date: _____

Name: A. Lakshmi Gayathri **Reg no:** 192411188

Q1. Sorting under different data conditions (n = 1000)

Prediction (before running)

Input	Selection	Bubble (early exit)	Insertion
Random	$n^2/2$ comparisons, about n swaps	about $n^2/2$ comparisons, about $n^2/4$ swaps	about $n^2/4$ comparisons and moves
Sorted	$n^2/2$ comparisons	$n-1$ comparisons	$n-1$ comparisons
Reverse	$n^2/2$ comparisons	$n^2/2$ comparisons and swaps	$n^2/2$ comparisons and moves

- Selection always scans the whole unsorted part, so its comparison count does not depend on the data.
- Bubble Sort can stop after a pass with no swaps, while Insertion Sort processes each element and places it in the sorted prefix. Insertion Sort shifts each element once per inversion; its comparisons also include terminating checks.

Datasets: random, reverse sorted (contrasting pair), nearly sorted (50 adjacent swaps), and 95% sorted plus a 5% unsorted new batch (edge case from the scenario).

Observed (comparisons / swaps or moves)

Dataset	Selection	Bubble	Insertion
Random	499,500 / 995	498,834 / 251,055	252,049 / 251,055
Sorted	499,500 / 0	999 / 0	999 / 0
Reverse	499,500 / 500	499,500 / 499,500	499,500 / 499,500
Nearly sorted	499,500 / 48	2,994 / 48	1,047 / 48

95% sorted + 5% batch	499,500 / 45	48,725 / 689	**1,688 / 689**

Prediction vs observation: every prediction was confirmed.

- Selection is flat at 499,500 comparisons in every case.
- Bubble needs about 3 full passes even on nearly sorted data, because an element moves only one position towards the front per pass.
- Insertion Sort performs one shift per inversion. Its comparison count equals the number of shifts plus up to n−1 terminating comparisons (fewer when an element moves all the way to the front).

Recommendation: Insertion Sort

- On the 95%-sorted data it uses 1,688 comparisons, against 48,725 for Bubble (29× more) and 499,500 for Selection (296× more).
- It is also stable and in-place.
- Selection's one advantage is the fewest writes (about n). That only matters if writes are expensive, which is not the case for priority scores.
- Weakness: on reverse-ordered data it degrades to $n^2/2$, so it should not be used if heavily disordered batches become common.

Q2. Sequential search

Comparisons (list of n tool records)

n	First	Middle	Last	Absent	Average (successful)
100	1	51	100	100	50.5
1,000	1	501	1,000	1,000	507
10,000	1	5,001	10,000	10,000	5,075
100,000	1	50,001	100,000	100,000	52,353

- Successful search costs about n/2 on average.
- Unsuccessful search always costs n.
- Cost is linear in list size.

Operating boundary

Upper bound: records × requests per second, against 10 comparisons per second.

Scenario	Comparisons per second	Share of one core
500 records, 10^3 requests/s	5×10	0.5%
10 records, 10 requests/s	10	100% (saturated)
2×10 records, 2×10 requests/s	4×10^1	400× over capacity

- **Reasonable choice:** a small list (hundreds of records), low request rate, and a list that changes constantly. A sequential list needs no ordering or index maintenance on issue and return.
- **Stop recommending it** when records × requests per second approaches 10 −10 , roughly 10 records at 10 lookups per second.

Changed condition (20 facilities)

Records and requests both grow about 20×, so total work grows about 400×. The original recommendation is withdrawn. Use a hash table keyed on Tool ID: O(1) insert on issue, O(1) delete on return, O(1) lookup. Trade-off: extra memory (Θ(n)) and no ordered traversal.

Q3. Brute-force string matching (text length 10,000, pattern length 10)

Test	Text / pattern	Predicted	Observed comparisons
Best	AAAA… / BAAAAAAAAA	1 comparison per alignment, so n−m+1	**9,991**
Worst	AAAA… / AAAAAAAAAB	m comparisons per alignment, so m(n−m+1)	**99,910**
Random (alphabet 4)	ACGT text / ACGTACGTAC	Mismatches come early, about 1.33 per alignment	**13,335**

Explanation

- The first test mismatches at the first character at every alignment.
- The second test matches 9 characters at every alignment and fails only on the last, so the work is repeated m times per alignment.
- Repeated characters in both text and pattern cause the repeated comparisons.
- Brute force never reuses what it learned from a partially matched prefix.
- All three tests have the same n and m, yet the work differs by 10×. Input size alone cannot predict cost: input structure matters, and the worst case is Θ(nm).

Q4. Closest pair (brute force)

Design

- Examine each unordered pair (i, j) with i < j. Skipping j < i avoids duplicates; skipping i = j avoids comparing a drone with itself.
- Number of unique pairs = C(n,2) = n(n−1)/2 (each of n drones pairs with n−1 others; dividing by 2 removes double counting).
- Compare squared distances to avoid the square root.
-

Results

n	Pairs examined	Distance calculations	n(n−1)/2	Time (ms, Python)
100	4,950	4,950	4,950	0.9
200	19,900	19,900	19,900	3.9
400	79,800	79,800	79,800	15.3
800	319,600	319,600	319,600	63.2
1,600	1,279,200	1,279,200	1,279,200	241
3,200	5,118,400	5,118,400	5,118,400	1,118

- Counts match the formula exactly. Doubling n gives 4× the pairs ($\Theta(n^2)$), and the times grow by about 4× per doubling.
- **Acceptability criterion:** 1 second per check in this environment. Brute force becomes impractical at about n ≈ 3,000 drones.
- **Source of inefficiency:** every pair is examined, even far-apart pairs that cannot be the closest pair.
- **A redesign must** avoid or reduce distant-pair checks, for example by ordering or partitioning points spatially so only nearby candidates are compared.

Q5. Convex hull (brute force)

Derived decision rule

1. Two points a = (x ,y) and b = (x ,y) define the line ax + by = c, with a = y −y , b = x −x , c = x y −y x .
2. For any other point k, the sign of s = (x −x)(y_k−y) − (y −y)(x_k−x) gives its side of the directed line a→b: positive is left, negative is right, zero is on the line.
3. Examine all n−2 remaining points.
4. Accept the directed edge (a,b) only if every other point has s ≥ 0 and no collinear point lies outside the segment ab. This keeps the endpoints extreme and avoids overlapping edges caused by collinear boundary points.
5. The accepted directed edges, collected in boundary order, form the convex-hull polygon.

Pseudocode

```
for each ordered pair (i,j), i != j:
   ok ← true
   for each k not in {i,j}:
      s ← cross(P[i], P[j], P[k])
      if s < 0: ok ← false; break
      if s = 0 and P[k] lies outside segment P[i]P[j]: ok ← false; break
   if ok: add edge (i,j)
```

Tests

- **Boundary plus interior:** 8 points (4 corners, 4 interior). Accepted edges: (0,0)→(10,0)→(10,10)→(0,10)→(0,0). Interior points are rejected.
- **Challenge case (collinear boundary points):** the square with extra midpoints. The rule accepts overlapping edges such as (0,0)→(5,0), (5,0)→(10,0) and (0,0)→(10,0), giving 12 edges instead of 4.
- **Fix:** accept an edge only if both endpoints are extreme. Reject a point that lies on the segment between two other points on the same line, or keep only the longest edge among collinear ones.

Growth

Side tests = $n(n-1)(n-2)$: 720 for n = 10, 6,840 for n = 20, 59,280 for n = 40, 492,960 for n = 80. This is $\Theta(n^3)$; doubling n gives about 8×.

Correctness argument

- If every other point lies on one closed side of the line through a and b, the line is a supporting line of the whole point set.
- A supporting line touching the set at a and b has the entire set inside one closed half-plane.
- If every point lies in the same closed half-plane and no collinear point lies beyond either endpoint of segment ab, then ab is a supporting boundary segment with extreme endpoints and belongs to the convex hull.
- If a point lies strictly on the other side, the line is not supporting and ab cannot be a convex-hull edge. If a collinear point lies beyond an endpoint, that endpoint is not extreme, so the candidate segment must also be rejected.

Q6. Exhaustive search (0/1 packages)

Design

- **Representation:** an n-bit mask; bit i = 1 means package i is loaded.
- **Generation:** all masks from 0 to 2^n-1.
- **Feasibility:** sum of weights ≤ W.
- **Evaluation:** sum of values.
- **Best retained:** keep the highest feasible value and its mask.

Correctness check

Weights [2,3,4,5], values [3,4,5,6], W = 5. Of the 16 subsets, 6 are feasible: ∅ (0), {0} (3), {1} (4), {2} (5), {3} (6), {0,1} (7). The best is {0,1} with value 7. The program returns 7 with mask `0b11`.

Growth

Prediction: 2^n candidates; each added package doubles the work.

n	Candidates examined	2^n	Time (s)
10	1,024	1,024	0.001
15	32,768	32,768	0.029
20	1,048,576	1,048,576	1.21
22	4,194,304	4,194,304	5.84

The prediction is validated.

Why a few extra packages explode the work: each new package is an independent in/out choice, so it doubles the solution space. Adding 10 packages multiplies the work by 2^{1} ≈ 1,000.

Deployment decision

With a fixed response time of 1 s in this Python environment, deploy the exhaustive search up to **n ≈ 19**; n = 20 already takes 1.21 s.

Q7. Diagnose, redesign, validate (selected: Q6 exhaustive search)

A. Diagnosis

- The algorithm examines 2^n candidate masks. For each mask, it scans up to n packages to recompute weight and value, so its running time is $\Theta(n \cdot 2^n)$, while the number of candidates is $\Theta(2^n)$.
- Work is repeated: every subset containing an already-overweight set is generated and rejected anyway, and each subset's weight and value are recomputed from scratch.
- Cause: the algorithm treats all subsets as independent candidates and never uses the fact that a partial selection exceeding W makes every extension infeasible.

B. Design objective

Reduce the candidate solutions examined from 2^n to a small fraction, with the same optimum, by exploiting partial-solution information.

C. Redesigned strategy (depth-first with pruning)

- Build solutions package by package, as a decision tree.
- Keep the running weight and value, so nothing is recomputed.
- **Prune 1:** if the running weight exceeds W, abandon the branch (all extensions are infeasible).
- **Prune 2:** compute an optimistic bound on the best value reachable from the branch. If it cannot beat the best solution so far, abandon the branch.
- Preprocessing: sort packages by value/weight ratio, which also tightens the bound.
- Cost: sorting (n log n), O(n) recursion depth, and an O(n) bound computation per node in the worst case.

```
sort packages by value/weight descending
```

```
best ← 0
search(k, w, v):
   best ← max(best, v)
   if k = n: return
   if bound(k, w, v) <= best: return
   if w + wt[k] <= W: search(k+1, w+wt[k], v+val[k])
   search(k+1, w, v)

bound(k, w, v):   // fill remaining capacity greedily, last item fractionally
```

D. Prediction

The nodes examined will drop dramatically below 2^n, with the gap widening as n grows. The optimum value will be identical to the exhaustive result.

E. Validation (same instances as Q6)

n	Exhaustive candidates	Redesign nodes	Same optimum	Exhaustive time (s)	Redesign time (s)
10	1,024	17	Yes	0.001	0.00002
15	32,768	57	Yes	0.029	0.00004
20	1,048,576	115	Yes	1.208	0.0001
22	4,194,304	36	Yes	5.84	0.00006

At larger sizes the redesign examined 77 nodes for n = 30 (against 2^3 ≈ 1.07×10), 226 for n = 40 and 210 for n = 50.

F. Evaluation

- The prediction is supported on the tested instances: the optimum is identical and node counts fall substantially, reaching 3–5 orders of magnitude fewer nodes in the larger cases. The improvement is instance-dependent.
- **Caveat:** node count depends on the instance. These instances use random, weakly correlated values and weights, which make the bound effective. For strongly correlated instances (value roughly proportional to weight) the bound prunes much less and the worst case is still exponential. Correlated instances were not tested, so the result should not be generalised to them.
- **Costs:** sorting preprocessing, per-node bound computation, and O(n) recursion memory.
- **When the original is preferable:** very small n (below about 15), where its simplicity, predictable work and trivial correctness outweigh the overhead.
- Conclusion: the redesign addresses the bottleneck for typical instances, but does not remove the worst-case exponential bound.

SOURCE CODE AND DATA

```
import random, time, itertools, math, json
random.seed(42)
R = {}

# ---------- Q1 sorting ----------
def selection(a):
    a=a[:]; c=s=0
    for i in range(len(a)-1):
        m=i
        for j in range(i+1,len(a)):
            c+=1
            if a[j]<a[m]: m=j
        if m!=i: a[i],a[m]=a[m],a[i]; s+=1
    return c,s
def bubble(a):
    a=a[:]; c=s=0; n=len(a)
    for i in range(n-1):
        sw=False
        for j in range(n-1-i):
            c+=1
            if a[j]>a[j+1]: a[j],a[j+1]=a[j+1],a[j]; s+=1; sw=True
        if not sw: break
```

```
    return c,s
def insertion(a):
    a=a[:]; c=s=0
    for i in range(1,len(a)):
        v=a[i]; j=i-1
        while j>=0:
            c+=1
            if a[j]>v: a[j+1]=a[j]; s+=1; j-=1
            else: break
        a[j+1]=v
    return c,s
n=1000
rnd=random.sample(range(n),n)
srt=list(range(n))
rev=srt[::-1]
near=srt[:];
for _ in range(50):
    i=random.randrange(n-1); near[i],near[i+1]=near[i+1],near[i]
ninefive=list(range(950))+random.sample(range(950,1000),50)  # 95% in place, 5% new batch appended
ninefive=sorted(ninefive[:950])+ninefive[950:]
sets={"Random":rnd,"Already sorted":srt,"Reverse sorted":rev,"Nearly sorted (50 adj swaps)":near,"95% sorted + 5% new batch":ninefive}
q1={}
for k,d in sets.items():
    q1[k]={}
    for name,f in (("Selection",selection),("Bubble",bubble),("Insertion",insertion)):
        t=time.perf_counter(); c,s=f(d); t=time.perf_counter()-t
        q1[k][name]=(c,s,round(t*1000,2))
```

```
R["Q1"]=q1


# ---------- Q2 sequential search ----------
def seq(a,k):
    c=0
    for x in a:
        c+=1
        if x==k: return c
    return c
q2={}
for n in (100,1000,10000,100000):
    a=list(range(n))
    q2[n]={"first":seq(a,0),"middle":seq(a,n//2),"last":seq(a,n-1),"absent":seq(a,-1),
           "avg_success":sum(seq(a,k) for k in random.sample(range(n),min(n,500)))/min(n,500)}
R["Q2"]=q2
# boundary: total comparisons/s at 1e8 cmp/s budget
q2b={}
for recs,req in ((500,1e3),(10000,1e4),(10000*20,2e5),(100000,1e6)):
    q2b[f"{recs} recs, {int(req)} req/s"]=recs*req/1e8
R["Q2_load"]=q2b


# ---------- Q3 string matching ----------
def bf(t,p):
    c=0;n=len(t);m=len(p);hits=0
    for i in range(n-m+1):
        j=0
        while j<m:
            c+=1
```

```
            if t[i+j]!=p[j]: break
            j+=1
        if j==m: hits+=1
    return c,hits
N=10000
tests={"Best: mismatch at first char":("A"*N,"B"+"A"*9),
       "Worst: repeated prefix, last mismatch":("A"*N,"A"*9+"B"),
       "Random log text (alphabet 4)":("".join(random.choice("ACGT") for _ in
range(N)),"ACGTACGTAC"),
       "Pattern matches everywhere":("A"*N,"A"*10)}
R["Q3"]={k:(bf(t,p),(len(t)-len(p)+1)) for k,(t,p) in tests.items()}

# ---------- Q4 closest pair ----------
def cp(pts):
    n=len(pts);pairs=d=0;best=float("inf")
    for i in range(n-1):
        for j in range(i+1,n):
            pairs+=1;d+=1
            v=(pts[i][0]-pts[j][0])**2+(pts[i][1]-pts[j][1])**2
            if v<best: best=v
    return pairs,d,best
q4={}
for n in (100,200,400,800,1600,3200):
    pts=[(random.random()*1e4,random.random()*1e4) for _ in range(n)]
    t=time.perf_counter(); p,d,b=cp(pts); t=time.perf_counter()-t
    q4[n]=(p,n*(n-1)//2,round(t*1000,1))
R["Q4"]=q4

# ---------- Q5 convex hull ----------
```

```
def hull(pts):
    n=len(pts);edges=[];tests=0
    for i in range(n):
        for j in range(n):
            if i==j: continue
            a,b=pts[i],pts[j];pos=neg=0
            for k in range(n):
                if k in(i,j): continue
                tests+=1
                v=(b[0]-a[0])*(pts[k][1]-a[1])-(b[1]-a[1])*(pts[k][0]-a[0])
                if v>0: pos+=1
                elif v<0: neg+=1
            if neg==0 : edges.append((i,j))   # all others on left/on line -> ccw edge
    return edges,tests
P1=[(0,0),(10,0),(10,10),(0,10),(5,5),(3,4),(7,2),(6,8)]
P2=[(0,0),(5,0),(10,0),(10,5),(10,10),(5,10),(0,10),(0,5),(5,5)]  # collinear boundary points
e1,_=hull(P1);e2,_=hull(P2)
R["Q5"]={"P1":[(P1[i],P1[j]) for i,j in e1],"P2":[(P2[i],P2[j]) for i,j in e2],
         "growth":{n:hull([(random.random(),random.random()) for _ in range(n)])[1] for n in
(10,20,40,80)}}


# ---------- Q6 knapsack exhaustive + Q7 redesign ----------
def exh(w,v,W):
    n=len(w);best=0;bs=0;cand=0
    for mask in range(1<<n):
        cand+=1
        tw=tv=0
        for i in range(n):
            if mask>>i&1: tw+=w[i];tv+=v[i]
```

```
        if tw<=W and tv>best: best=tv;bs=mask
    return best,bs,cand
def bnb(w,v,W):
    n=len(w);order=sorted(range(n),key=lambda i:-v[i]/w[i])
    ws=[w[i] for i in order];vs=[v[i] for i in order]
    best=[0];nodes=[0]
    def bound(k,cw,cv):
        r=W-cw;b=cv
        for i in range(k,n):
            if ws[i]<=r: r-=ws[i];b+=vs[i]
            else: b+=vs[i]*r/ws[i];break
        return b
    def rec(k,cw,cv):
        nodes[0]+=1
        if cv>best[0]: best[0]=cv
        if k==n: return
        if bound(k,cw,cv)<=best[0]: return
        if cw+ws[k]<=W: rec(k+1,cw+ws[k],cv+vs[k])
        rec(k+1,cw,cv)
    rec(0,0,0);return best[0],nodes[0]
# small verifiable instance
w=[2,3,4,5];v=[3,4,5,6];W=5
ex=exh(w,v,W)
feas=[(sum(w[i] for i in c),sum(v[i] for i in c),c) for r in range(5) for c in
itertools.combinations(range(4),r) if sum(w[i] for i in c)<=W]
R["Q6_small"]={"exh":ex[:1]+(bin(ex[1]),ex[2]),"feasible":feas,"bnb":bnb(w,v,W)}
q6={};q7={}
for n in (10,15,20,22):
```

```
    ww=[random.randint(5,40) for _ in range(n)];vv=[random.randint(10,100) for _ in range(n)];WW=sum(ww)//2
    t=time.perf_counter();b,bs,c=exh(ww,vv,WW);te=time.perf_counter()-t
    t=time.perf_counter();b2,nd=bnb(ww,vv,WW);tb=time.perf_counter()-t
    q6[n]=(2**n,c,b,round(te,3))
    q7[n]=(c,nd,b==b2,round(te,3),round(tb,5))
R["Q6"]=q6;R["Q7"]=q7
# BnB at larger n
big={}
for n in (30,40,50):
    ww=[random.randint(5,40) for _ in range(n)];vv=[random.randint(10,100) for _ in range(n)]
t=time.perf_counter();b,nd=bnb(ww,vv,sum(ww)//2);big[n]=(nd,2**n,round(time.perf_counter()-t,4))
R["Q7_big"]=big
print(json.dumps(R,indent=1,default=str))
```